



Bikkad IT Institute

16. SQL Window Ranking Functions

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

SQL Window Ranking Functions

16 · Window Ranking — Complete Reference Guide

Table of Contents

1. ROW_NUMBER
2. RANK
3. DENSE_RANK
4. NTILE
5. CUME_DIST

1 – 3. ROW_NUMBER, RANK, DENSE_RANK

→ *Task 1: Rank orders by Sales — compare all three ranking functions*

```
SELECT
    OrderID, ProductID, Sales,
    ROW_NUMBER() OVER (ORDER BY Sales DESC) AS SalesRank_Row,
    RANK() OVER (ORDER BY Sales DESC) AS SalesRank_Rank,
    DENSE_RANK() OVER (ORDER BY Sales DESC) AS SalesRank_Dense
FROM Sales.Orders;
```

→ *Task 2: Top-N Analysis — Highest sale for each product*

```
SELECT *
FROM (
    SELECT
        OrderID, ProductID, Sales,
        ROW_NUMBER() OVER
            (PARTITION BY ProductID ORDER BY Sales DESC) AS RankByProduct
    FROM Sales.Orders
) AS TopProductSales
WHERE RankByProduct = 1;
```

→ *Task 3: Bottom-N Analysis — Lowest 2 customers by total sales*

```
SELECT *
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```

FROM (
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales,
        ROW_NUMBER() OVER (ORDER BY SUM(Sales)) AS RankCustomers
    FROM Sales.Orders
    GROUP BY CustomerID
) AS BottomCustomerSales
WHERE RankCustomers <= 2;

```

→ *Task 4: Assign unique IDs to rows in OrdersArchive*

```

SELECT
    ROW_NUMBER() OVER (ORDER BY OrderID, OrderDate) AS UniqueID,
    *
FROM Sales.OrdersArchive;

```

→ *Task 5: Remove duplicate rows — keep latest by CreationTime*

```

SELECT *
FROM (
    SELECT
        ROW_NUMBER() OVER
            (PARTITION BY OrderID ORDER BY CreationTime DESC) AS rn,
        *
    FROM Sales.OrdersArchive
) AS UniqueOrdersArchive
WHERE rn = 1;

```

4. NTILE — Divide Rows into Buckets

→ *Task 6: Divide orders into 1, 2, 3, and 4 buckets by Sales*

```

SELECT
    OrderID, Sales,
    NTILE(1) OVER (ORDER BY Sales) AS OneBucket,
    NTILE(2) OVER (ORDER BY Sales) AS TwoBuckets,
    NTILE(3) OVER (ORDER BY Sales) AS ThreeBuckets,
    NTILE(4) OVER (ORDER BY Sales) AS FourBuckets,
    NTILE(2) OVER (PARTITION BY ProductID ORDER BY Sales) AS TwoBucketByProducts
FROM Sales.Orders;

```

→ *Task 7: Segment orders into High / Medium / Low Sales categories*

```

SELECT
    OrderID, Sales, Buckets,
    CASE
        WHEN Buckets = 1 THEN 'High'
        WHEN Buckets = 2 THEN 'Medium'
        WHEN Buckets = 3 THEN 'Low'
    END AS SalesSegmentations

```

OUR LOCATIONS

```
FROM (  
    SELECT  
        OrderID, Sales,  
        NTILE(3) OVER (ORDER BY Sales DESC) AS Buckets  
    FROM Sales.Orders  
    ) AS SalesBuckets;
```

→ *Task 8: Divide orders into 5 processing groups*

```
SELECT  
    NTILE(5) OVER (ORDER BY OrderID) AS Buckets,  
    *  
FROM Sales.Orders;
```

5. CUME_DIST — Cumulative Distribution

→ *Task 9: Products that fall within the highest 40% of prices*

```
SELECT  
    Product, Price, DistRank,  
    CONCAT(DistRank * 100, '%') AS DistRankPerc  
FROM (  
    SELECT  
        Product, Price,  
        CUME_DIST() OVER (ORDER BY Price DESC) AS DistRank  
    FROM Sales.Products  
    ) AS PriceDistribution  
WHERE DistRank <= 0.4;
```